

Table des matières

Mini-cas — La Brûlerie, un mois de caisse du fichier au contrat	1
Étape 0 — Le café, et ce que vous allez bâtir	1
Étape 1 — Le projet, à côté de celui de Boréal	2
Étape 2 — Le premier regard	3
Étape 3 — La RAW, et son journal	4
Étape 4 — Le bronze de la caisse	5
Étape 5 — Une porte d'entrée, et la fenêtre	6
Étape 6 — Le torréfacteur, en petit terrain	7
Étape 7 — Le contrat de la caisse	8
Étape 8 — La preuve : les totaux du mois	9
Récapitulatif	10
Annexe — L'arborescence et les commandes	11

Mini-cas — La Brûlerie, un mois de caisse du fichier au contrat

Mini-cas figé — Formation Data Engineering · les démarches des Cours 01 à 06, 08 et 11, en petit

Étape 0 — Le café, et ce que vous allez bâtir

La Brûlerie du Passage est un café de quartier à Montréal. Elle torréfie son café sur place, sert au comptoir et n'a qu'une caisse. Le mini-cas en prend un mois, juin 2025, figé en fichiers dans `Data_Domain/_univers/brulerie/mini_cas/sources/` : il n'y a rien à générer, les données sont versionnées avec le parcours.

Vous allez refaire seul, sur ce mois, l'arc que Boréal vous a fait parcourir pièce par pièce :

les fichiers livrés → RAW, avec son journal des chargements → bronze de la caisse, en Parquet par jour → argent du torréfacteur → contrat de la caisse, le tout lancé par une seule commande.

Ce document ne réexplique rien. Chaque étape nomme la section de cours qui fonde son geste, et ne garde que ce qui est propre à ce cas. Trois choses le sont : un jour que la caisse a exporté deux fois, et que le bronze ne doit compter qu'une ; un torréfacteur dont le journal connaît tout le mois d'avance, alors que ses courbes arrivent jour après jour ; et la taille du commerce, qui n'a besoin ni d'Airflow, ni de Kafka, ni de Spark, ni d'Iceberg.

Il se fait une fois le Cours 12 derrière vous, avant la synthèse Aurore. Comptez une séance, deux heures et demie environ : un peu plus d’une heure de lecture, mise en situation comprise, et une heure vingt pour écrire le dépôt, dont la plus grande part sans bloc à recopier.

Dossier des sources	Fichiers	Ce qu’il porte
caisse/2025-06/	62	un export par jour d’ouverture, les tickets et leurs lignes, plus l’export repris du 12 juin
torrefaction/	9	le journal des 47 lots du mois, et une courbe par jour de torréfaction, 8 jours
catalogue/	2	les 32 produits, et le journal de leurs modifications
fidelite/	2	les 994 membres, et le journal de leurs consentements
meteo/	1	les 30 jours de la station de l’aéroport de Montréal, réels

Le dossier documents/ et le dossier graines/ ne servent pas ici : le premier est un corpus pour AI Engineering, le second une copie à plat que le mini-cas d’Analytics Engineering charge d’un coup. Le sens de chaque colonne est dans sources/dictionnaire_brulerie.md.

Le corrigé est dans Solution/01_de/, le dépôt tel qu’il doit être à la fin. Chaque bloc de ce document qui appartient au dépôt en est tiré, sans ses commentaires d’en-tête.

Étape 1 — Le projet, à côté de celui de Boréal

La configuration hors du code est au Cours 05 (Theorie_05.md, section 2), et l’empaquetage en une seule porte d’entrée à la section 7.

Le mini-cas vit dans Data_Domain/Projet_Brulerie/, que Git ignore comme Projet_Boreal/. C’est aussi le dossier du mini-cas d’Analytics Engineering : les deux dépôts s’y rangent côte à côte, 01_de et 02_ae, chacun avec sa base. Depuis l’Anaconda Prompt, env formations actif :

```
cd Data_Domain
mkdir Projet_Brulerie\01_de
cd Projet_Brulerie
xcopy ..\univers\brulerie\mini_cas\sources sources\ /E /I /Q
cd 01_de
git init
```

xcopy doit annoncer 101 fichiers copiés : les 100 du mois et leur fichier d’empreintes. Le projet prend la forme de Boréal, en petit :

Dossier	Ce qu'il porte	Qui l'écrit
Projet_Brulerie/sources/	le mois figé, jamais retouché	vous, une fois, par xcopy
Projet_Brulerie/plateforme/lac/bronze/caisse/	la caisse en Parquet, une partition par jour	le pipeline
Projet_Brulerie/plateforme/entrepot/brulerie.duckdb	les schémas raw, argent et pipeline	le pipeline
Projet_Brulerie/01_de/	le dépôt : le paquet brulerie, ses scripts SQL, son test	vous

Le dépôt porte un config.toml qui dit où tout se range, et le code n'en connaît aucun chemin :

```
[chemins]
sources = "sources"
lac = "plateforme/lac"
entrepot = "plateforme/entrepot/brulerie.duckdb"
```

Et un paquet, brulerie, bâti comme pipeline chez Boréal : une porte d'entrée, __main__.py, un module par étape, et config.py, qui lit ce fichier par tomlib et retrouve le projet, le parent du dépôt.

Étape 2 — Le premier regard

DuckDB pour regarder sans charger est au Cours 01 (Theorie_01.md, section 6), et le type deviné à la section 7.

Avant d'écrire une ligne de pipeline, regardez ce que la caisse livre. Dans la ligne de commande DuckDB, lancée depuis Projet_Brulerie :

```
describe select * from read_csv('sources/caisse/2025-06/tickets_*.csv');
```

DuckDB devine, et se trompe là où le Cours 01 l'annonçait :

Colonne	Deviné	Ce qu'elle est
sous_total, tps, tvq, total, arrondi, encaisse	DOUBLE	des montants au cent
ticket_origine	VARCHAR	un numéro de ticket ; le premier fichier n'en porte aucun, il devine sur du vide
horodatage	TIMESTAMP	juste : l'heure de Montréal, sans décalage

Le DOUBLE ne se voit qu'au total. La somme de l'encaissé des tickets payés, sur le dernier export de chaque jour, sort à 18666.379999999997, puis à 18666.380000000001 à l'exécution suivante, et parfois à 18666.38 : DuckDB additionne sur plusieurs fils, l'ordre des additions change d'une fois à l'autre, et avec lui le dernier chiffre d'un nombre à virgule flottante. Le Cours 08 a rencontré le même effet sur ses moyennes horaires (Theorie_08.md, section 9). Un total qui change sans que les données changent ne se compare à rien.

Puis cherchez ce que le mois a de bizarre. Le nom de chaque fichier porte son jour et l'instant de son export, tickets_2025-06-12_202506130705.csv ; comptez-les par jour :

```
select regexp_extract(filename, '_(\d{4}-\d{2}-\d{2})_', 1) as jour,
       count(distinct filename) as exports,
       count(*) as lignes,
       count(distinct ticket_id) as tickets
from read_csv('sources/caisse/2025-06/tickets_*.csv', filename = true)
group by 1
having count(distinct filename) > 1;
```

Une seule ligne : le 2025-06-12, 2 exports, 123 lignes pour 65 tickets. La caisse a exporté la journée à 15 h 42, avec 58 tickets, puis de nouveau le lendemain à 7 h 05, avec les 65 de la journée entière. Le second contient le premier. C'est le piège de ce mois, et tout le pipeline est construit pour qu'il ne compte pas deux fois.

Le mois porte aussi 15 tickets annulés et 1 remboursement, à côté de 1 511 ventes payées : le contrat de l'étape 7 saura quoi en faire.

Étape 3 — La RAW, et son journal

Le chargement complet et l'incrémental sont au Cours 03 (Theorie_03.md, section 2), l'idempotence et le journal des chargements à la section 3, la couche RAW à la section 4.

Le chargeur est celui de Boréal, à peu de chose près : une empreinte SHA-256 par fichier, un journal pipeline.chargements, une transaction qui écrit la table et le journal ensemble, et tout en texte avec les trois colonnes de lignage. Ce qui change, c'est la liste des livraisons, et la façon de lire un lot dans un nom de fichier :

```
LIVRAISONS = [
  ("caisse_tickets", "sources/caisse/*/tickets_*.csv"),
  ("caisse_lignes", "sources/caisse/*/lignes_*.csv"),
  ("torrefaction_courbes", "sources/torrefaction/courbes/*/courbe_*.csv"),
  ("torrefaction_lots", "sources/torrefaction/lots.csv"),
  ("catalogue_produits", "sources/catalogue/produits.csv"),
  ("catalogue_modifications", "sources/catalogue/modifications.csv"),
  ("fidelite_membres", "sources/fidelite/membres.csv"),
  ("fidelite_consentements", "sources/fidelite/consentements.csv"),
  ("meteo_quotidien", "sources/meteo/montreal_quotidien.csv"),
```

```

]

# Le lot se lit dans le nom : le jour, suivi de l'instant de l'export pour la caisse.
LOT = re.compile(r"_(\d{4}-\d{2}-\d{2})(?:_\d{12})?)"

```

Un export de la caisse a pour lot 2025-06-12_202506130705, une courbe 2025-06-03, et un fichier sans date dans son nom, lots.csv ou produits.csv, le lot complet : il est livré en entier, et c’est son empreinte qui dit s’il a changé.

La RAW garde les deux exports du 12 juin, chacun sous son lot, parce qu’elle garde ce qui a été livré. Mesuré :

Lot	Tickets dans raw.caisse_tickets
2025-06-12_202506121542	58
2025-06-12_202506130705	65

Sur le mois, raw.caisse_tickets porte 1 585 lignes pour 1 527 tickets, et raw.caisse_lignes 2 626 lignes : les 58 tickets en trop sont ceux du premier export du 12. Aucune colonne n’y est autre chose que du texte, hors l’heure de chargement. Tirer l’état courant appartient à celui qui lit la RAW, ici le bronze de l’étape suivante, chez Boréal le STAGING d’Analytics Engineering.

À retenir — raw.fidelite_membres porte le prénom, le nom, le courriel et le téléphone des 994 membres. La RAW garde ce qu’on lui livre, données personnelles comprises, comme le rappelle le Cours 03, section 4 ; les membres sont synthétiques, mais la règle est la même que chez Boréal.

Étape 4 — Le bronze de la caisse

La règle du bronze est au Cours 02 (Theorie_02.md, section 4), le partitionnement à la section 5 et DuckDB sur le lac à la section 6. Tester une transformation est au Cours 05, section 6.

La caisse est le seul flux quotidien du café, et c’est elle qui va au lac. Une partition par jour, jour=2025-06-12/, parce que le jour est à la fois le grain de livraison, le filtre fréquent et une cardinalité raisonnable : trente partitions pour le mois. La partition suit le jour livré, et un jour livré de nouveau remplace sa partition, rien d’autre.

Choisir l’export d’un jour est une décision, et une décision se teste seule. C’est une fonction pure, qui prend des noms et rend un nom :

```

def dernier_export(noms: list, jour: str):
    """Le dernier export d'un jour parmi des noms de fichiers, ou None si le café était fermé.

    L'instant de l'export est dans le nom, AAAAMMJJHHMM : il se trie comme du texte.
    """

```

```
du_jour = [(m.group(3), nom) for nom in noms if (m := EXPORT.match(nom)) and m.group(2) == jour]
return max(du_jour)[1] if du_jour else None
```

Le test, tests/test_bronze.py, vérifie quatre cas : le second export du 12 l’emporte, l’ordre des noms ne compte pas, un export fait le 13 au matin appartient au 12 par son nom, et un jour sans export n’en rend aucun. Depuis 01_de, pytest rend 4 passed, en quelques centièmes de seconde.

Puis l’écriture. Les types sont déclarés, jamais devinés : les montants en DECIMAL(10,2), l’horodatage en TIMESTAMP, qui ne perd rien puisque le fichier n’a pas de fuseau, et ticket_origine en BIGINT. La partition se vide avant d’être réécrite :

```
partition = lac / "bronze" / "caisse" / quoi / f"jour={jour}"
# Remplacer la partition, c'est la vider d'abord : un fichier qui y traînerait
# serait relu avec elle. Puis COPY crée son fichier, pas les dossiers du chemin.
if partition.exists():
    shutil.rmtree(partition)
partition.mkdir(parents=True)
source = literal((dossier / nom).as_posix())
cible = literal((partition / f"{quoi}.parquet").as_posix())
types = ", ".join(f"'{c}': '{t}'" for c, t in colonnes.items())
con.execute(f"""
    COPY (SELECT *, {source} AS _fichier
          FROM read_csv({source}, header = true, columns = {{{types}}}))
    TO {cible} (FORMAT parquet, COMPRESSION zstd)""")
```

Vider la partition n’est pas une précaution de style. Un COPY vers un fichier écrase ce fichier-là et rien d’autre : un second fichier déposé à côté, par un essai ou une ancienne version du code, serait relu avec la partition par tout lecteur du lac. L’étape 7 le mesure.

Mesuré sur le mois : 30 partitions de tickets et 30 de lignes, 1 527 tickets et 2 532 lignes, 220 Ko de fichiers pour tout le lac. La partition du 12 porte 65 tickets, tirés de l’export de 7 h 05. Le schéma d’un fichier, lu par parquet_schema, montre les montants en DecimalType(scale=2, precision=10) et l’horodatage sans ajustement à l’UTC.

Étape 5 — Une porte d’entrée, et la fenêtre

La porte d’entrée unique est au Cours 05 (Theorie_05.md, section 7). L’intervalle de données est au Cours 06 (Theorie_06.md, section 3), le rattrapage à la section 4, la fenêtre, le journal et la reconstruction à la section 5, et ce qu’un ordonnanceur fait de plus qu’une tâche planifiée à la section 1.

Une exécution enchaîne les quatre étapes, dans l’ordre, sur la connexion que la porte d’entrée a ouverte une fois :

```
def une_execution(con, fenetre) -> None:
    debut = time.monotonic()
    LOG.info("exécution %s", fenetre)
```

```
raw.executer(con, fenetre)
bronze.executer(con, fenetre, chemin("sources"), chemin("lac"))
argent.executer(con)
controles.executer(con)
LOG.info("fin, en %.1f s", time.monotonic() - debut)
```

La commande reçoit sa tranche, elle ne la lit jamais sur l'horloge :

```
python -m brulerie --depuis 2025-06-12 --jusqua 2025-06-12
```

La fenêtre ne regarde que des noms. Un lot tombe dans la tranche par son jour, et un fichier livré en entier y tombe toujours, le journal décidant ensuite s'il a changé :

```
def contient(self, lot: str) -> bool:
    """Le lot d'un fichier tombe-t-il dans la tranche ? « 2025-06-12_202506130705 » se lit par
    son jour ; « complet » y tombe toujours."""
    if lot == "complet":
        return True
    return self.depuis <= date.fromisoformat(lot[:10]) <= self.jusqua
```

Le café n'a pas d'Airflow, et n'en a pas besoin : une commande par jour, que le Planificateur de tâches de Windows ou cron lancent chaque matin sur la veille. Ce qu'ils n'ont pas, c'est la mémoire de ce qu'ils ont manqué, et c'est ce qui les distingue d'un ordonnanceur. L'option `--par-jour` en tient lieu : une exécution par jour de la tranche, dans l'ordre, comme un rattrapage.

Le rattrapage n'est une reprise que s'il redonne ce qu'un chargement d'un bloc aurait donné. Mesuré sur deux projets neufs :

Chargement	Commande	Durée
jour par jour, 30 exécutions	<code>--depuis 2025-06-01 --jusqua 2025-06-30 -</code> <code>-par-jour</code>	6 s
d'un bloc, une exécution	<code>--depuis 2025-06-01 --jusqua 2025-06-30</code>	2,3 s

Comparés table par table, heure de chargement exclue, les deux projets sont identiques : 11 tables et 10 327 lignes dans l'entrepôt, 60 fichiers et 4 059 lignes dans le bronze, 0 écart. Relancer le mois sur un projet déjà chargé ne charge rien dans la RAW, réécrit le bronze à l'identique et prend 0,9 s. Rejouer le 12 juin seul prend 0,2 s, et redonne ses 65 tickets.

Étape 6 — Le torréfacteur, en petit terrain

La chaîne qui produit un point est au Cours 08 (Theorie_08.md, section 1), la série régulière à la section 9, et la règle propre à chaque nature de point à la section 10.

Le torréfacteur a deux sondes, dans le grain et dans l'air, lues toutes les 10 secondes pendant un lot, et un logiciel qui marque quatre événements : la charge, le point bas, le premier craquement et la sortie. Il garde deux traces : les courbes, un fichier par jour de torréfaction, et le journal des lots, où il déclare pour chacun la seconde du premier craquement, la durée et la température de sortie. Deux natures de points dans un même fichier : des températures, qui se lisent en continu, et des événements, qui se lisent par leur seconde.

Le mois compte 8 jours de torréfaction, 47 lots et 3 718 lectures. Il est propre : un pas de 10 secondes partout, aucune sonde vide. Le schéma argent en tire une ligne par lot, avec deux mesures du métier : le temps de développement, la part de la torréfaction qui suit le premier craquement, et la perte de poids du grain. Le lot L-2025-0214 se refait de tête : 780 secondes, premier craquement à 660, donc 120 secondes de développement, 15,4 % ; 3,00 kg de vert pour 2,58 kg torréfiés, 14,0 % de perte ; et 79 lectures, une toutes les 10 secondes de 0 à 780.

Profil	Lots	Durée	Sortie	Développement	Perte
CLAIRE	3	11,3 min	206,3 °C	12,7 %	13,2 %
MOYENNE	26	12,5 min	211,5 °C	17,8 %	14,9 %
MOYENNE_FONCEE	18	14,0 min	217,4 °C	23,6 %	15,9 %

Plus un café est foncé, plus on le laisse se développer après le premier craquement, et plus il perd d'eau et de poids : les trois colonnes montent ensemble.

Le témoin recoupe ce que le logiciel déclare avec ce que ses sondes ont enregistré :

```
SELECT count(*) AS lots,
       count(*) FILTER (WHERE a.duree_s IS NULL
                          OR a.premier_craquement_s <> j.premier_craquement_s::INTEGER
                          OR a.duree_s <> j.duree_s::INTEGER
                          OR a.temp_sortie IS DISTINCT FROM j.temp_sortie::DECIMAL(5, 1)
                          OR a.lectures <> a.duree_s / 10 + 1) AS ecart
FROM raw.torrefaction_lots AS j
JOIN argent.torrefaction_lots AS a USING (lot_id);
```

Attendu : 47 lots, 0 écart. Le mois n'a pas de sonde en panne, et le témoin le prouve ; le jour où la sonde du grain se tait à la sortie d'un lot, la température de sortie de la courbe est vide, et le lot sort en écart.

Piège — un journal complet devant des courbes qui arrivent jour par jour. Le journal des lots est un export complet : dès le 1er juin, il connaît les 47 lots du mois. Les courbes, elles, arrivent avec leur jour. Écrit naïvement, le témoin s'arrête le 1er juin sur une table de courbes qui n'existe pas encore, mesuré, puis compterait le 3 juin en écart les 40 lots des jours à venir. La règle juste : un lot est dû quand son jour est déjà livré. Le dernier jour livré se lit dans le journal des chargements, et non dans la fenêtre de l'exécution, sans quoi rejouer le 12 juin après le mois ferait disparaître de l'argent les lots du 13 au 30.

Étape 7 — Le contrat de la caisse

Le pipeline qui ne plante pas et qui ment est au Cours 11 (Theorie_11.md, section 1), le contrat à la section 2, les dimensions d'une règle à la section 3, et le choix de bloquer, d'avertir ou de suivre à la section 4.

Le dictionnaire de la caisse dit comment elle calcule : la TPS à 5 % et la TVQ à 9,975 % des lignes taxables, chacune arrondie au cent ; le total ; le comptant arrondi aux 5 cents ; un ticket annulé qui n'encaisse rien ; un remboursement qui vise une vente payée d'un jour précédent. Le contrat recalcule chacune de ces règles sur tout le bronze, sql/controles_caisse.sql, neuf en tout, et rend une ligne par règle avec son nombre d'écarts. La première est celle qui compte le plus ici :

```
SELECT 'numéros de tickets uniques' AS regle, count(*) - count(DISTINCT ticket_id) AS ecarts
FROM tickets
```

Un jour chargé deux fois se trahit par ses numéros. Sur les deux exports du 12 juin lus ensemble, la règle compte 58 écarts : ce sont les 58 tickets du premier export, revenus dans le second.

Toutes les règles bloquent : un ticket compté deux fois ou une taxe mal recalculée est un chiffre faux, pas un avertissement. Le pipeline s'arrête avec le code 1 et nomme chaque règle rompue. Mesuré en déposant le premier export du 12 dans la partition du 12, à côté du second, puis en lançant le 13 juin :

```
ERROR      règle rompue : numéros de tickets uniques, 58 écart(s)
ERROR      1 règle(s) du contrat de la caisse en écart.
```

Le contrat juge tout le bronze, et non la seule tranche de l'exécution : c'est le 13 qui l'a vu, alors que le défaut dormait dans le 12. Rejouer le 12 juin répare, parce que la partition se vide avant d'être réécrite, et le contrat repasse à neuf règles sans écart.

Sur le mois chargé normalement : neuf règles, aucun écart, à chacune des 30 exécutions. La règle du remboursement demande un ordre : le remboursement du 27 vise une vente du 25, et un 27 chargé avant le 25 sortirait en écart. C'est une dépendance entre les jours, et le rattrapage la respecte en les prenant dans l'ordre.

Étape 8 — La preuve : les totaux du mois

Dans la ligne de commande DuckDB, depuis Projet_Brulerie, le bronze se lit comme au Cours 02 :

```
with t as (
  select * from read_parquet('plateforme/lac/bronze/caisse/tickets/**/*.parquet',
    hive_partitioning = true)
)
select count(*) as tickets,
count(*) filter (where statut = 'PAYE' and type_ticket = 'VENTE') as ventes_payees,
count(*) filter (where statut = 'ANNULE') as annules,
sum(sous_total) filter (where statut = 'PAYE') as ventes_avant_taxes,
sum(tps) filter (where statut = 'PAYE') as tps,
```

```
sum(tvq) filter (where statut = 'PAYE') as tvq,
sum(encaisse) filter (where statut = 'PAYE') as encaisse
from t;
```

Mesure	Valeur
Tickets	1 527
Ventes payées, et un remboursement	1 511, et 1
Annulés	15
Ventes avant taxes	16 718,00 \$
TPS, TVQ	651,55 \$, 1 295,66 \$
Encaissé	18 666,38 \$

Ce sont les chiffres que le générateur a écrits pour ce mois dans `_univers/brulerie/CHIFFRES_ATTENDUS.md`, section 2.1, au cent. L'encaissé tombe juste, et le même à chaque exécution, là où la lecture devinée de l'étape 2 changeait de dernier chiffre d'une fois à l'autre : la seule différence est le type déclaré au bronze. Un DECIMAL s'additionne en entiers de cents, et l'ordre des additions n'y change rien.

Récapitulatif

Démarche	Où elle est enseignée	Où ici
la configuration hors du code, la porte d'entrée unique	Cours 05, sections 2 et 7	Étapes 1 et 5
regarder avant de charger, le type deviné	Cours 01, sections 6 et 7	Étape 2
la RAW, son lignage, son journal et ses empreintes	Cours 03, sections 2 à 4	Étape 3
le lot lu dans un nom de fichier	ce mini-cas	Étape 3
la règle du bronze, le partitionnement par jour	Cours 02, sections 4 à 6	Étape 4
le dernier export d'un jour, et la partition vidée avant d'être réécrite	ce mini-cas	Étape 4

Démarche	Où elle est enseignée	Où ici
une décision testée seule	Cours 05, section 6	Étape 4
l'intervalle de données, le rattrapage, la fenêtre et le journal	Cours 06, sections 3 à 5	Étape 5
une série à pas fixe, deux natures de points	Cours 08, sections 1, 9 et 10	Étape 6
le lot dû, devant un journal complet	ce mini-cas	Étape 6
un contrat qui bloque, jugé sur tout le bronze	Cours 11, sections 1 à 4	Étape 7

Ce que le mini-cas ne fait pas, parce qu'un café n'en a pas besoin :

Ce qui manque	Où le trouver
l'ordonnanceur, ses exécutions et ses tâches	Cours 06 : une tâche planifiée du système suffit à une commande par jour
la base par ODBC, l'API REST	Cours 04 : la caisse exporte des fichiers, et la météo est déjà là
Spark	Cours 07 : 4 059 lignes de bronze tiennent dans n'importe quel portable
MQTT et Kafka	Cours 09 : les courbes du torréfacteur arrivent en fichiers, et personne ne les lit en direct
Iceberg et Delta	Cours 10 : trente partitions n'ont ni schéma qui change, ni lecteurs concurrents
les secrets, le conteneur	Cours 05, sections 3 et 8 : aucun serveur, aucun mot de passe
le journal des exécutions	Cours 05, section 4 : le journal de texte suffit à un seul lecteur

Le même pipeline que Boréal, réduit à un mois et à un commerce. Tout ce qui s'y charge se recompte contre la caisse, et un ticket s'y suit de tête, du fichier jusqu'au bronze.

Annexe — L'arborescence et les commandes

```
Projet_Brulerie/
├─ sources/
├─ plateforme/
```

le mois figé, copié par xcopy

	lac/bronze/caisse/	tickets/ et lignes/, une partition par jour
	entrepot/brulerie.duckdb	raw, argent, pipeline
	01_de/	le dépôt du mini-cas
	config.toml	les chemins
	pytest.ini	
	.gitignore	
	brulerie/	__main__, config, fenetre, entrepot, raw, bronze, argent, controles
	sql/	argent_torrefaction.sql, controles_caisse.sql
	tests/test_bronze.py	

Pour tout rejouer, depuis Projet_Brulerie/01_de, env formations actif :

```
pytest
python -m brulerie --depuis 2025-06-01 --jusqua 2025-06-30 --par-jour
```

Le corrigé complet, commentaires compris, est dans Solution/01_de/. Son README.md dit comment le rejouer sans rien retaper.